6/8 – Software Engineering Introduction

Right Software at the Right Time

Core Concept

- Primary goal of software engineering is delivering the right software at the right time
- Both quality and timing require continuous negotiation between teams

Wrong Software

- Does not solve user needs
- Lacks critical functionality
- Is overly complex for users
- Is too simplistic for the problem being solved
- Contains security weaknesses
- Reflects internal preferences rather than customer needs

Wrong Time

- Ships too late to capture market opportunities
- Ships too early with insufficient quality
- Launches before critical functionality is ready
- Example – Explore Data launched without saved filters to meet timeline goals; acceptable tradeoff for initial release; failure to prioritize saved filters immediately afterward created customer pain points; demonstrates how timing decisions continue after launch

Scope & Capacity

Scope

- **Definition** – amount of work included within a feature, project, or release
- Primary lever when deadlines cannot change
- Reducing scope is prioritization, not failure
- Focus should remain on critical functionality
- Example – cleaning a bedroom vs cleaning an entire house: same amount of time available, different amount of work completed

Capacity

- **Definition** – amount of work a team can realistically complete within a given timeframe
- More than just team size – includes throughput, coordination, and experience
- Additional people create communication overhead
- Increased contributors can reduce consistency and quality
- Capacity is negotiated, not assumed
- Engineers should push back when expectations exceed realistic delivery capability
- Successful delivery requires balancing scope and capacity

SEA Scope Negotiation Example

- **Included in MVP:**
  - Administrative dashboards
  - Key filters (Housing, Classroom)
  - Student engagement metrics
  - At-risk student reporting
- **Deferred to Parking Lot:**

  - Interactive campus map
  - Student-facing dashboards
- **Key lessons:**
  - High-effort features can be replaced with simpler alternatives during MVP development
  - Deferred features remain candidates for future releases
  - Product, Engineering, and customer feedback collectively influence prioritization decisions

Software Development Life Cycle (SDLC)

Analysis

- **Purpose** – understand user needs and business requirements
- **Activities** – customer interviews, surveys, stakeholder meetings, requirements gathering
- **Outputs** – user stories, use cases, requirements documentation

Design

- **Purpose** – transform requirements into technical solutions
- **Activities** – architecture planning, tool selection, system design, diagram creation
- **Outputs** – design documents, technical diagrams, architecture plans

Development

- **Purpose** – build the solution
- **Activities** – write code, conduct peer reviews, implement functionality
- **Key principle** – code review is an integral part of development, not a separate activity

Testing & QA

- **Purpose** – validate quality and requirements
- **Activities** – unit testing, manual testing, requirement validation, regression testing
- **Goal** – ensure software functions as intended and continues working as new code is added

Deployment

- **Purpose** – release software to users
- **Activities** – publish release notes, execute deployment plans, perform staged rollouts when appropriate

Maintenance

- **Purpose** – support software after release
- **Activities** – bug fixes, feature enhancements, performance improvements, version updates
- **Important note** – the SDLC is cyclical rather than linear; feedback from maintenance informs future analysis and planning

Scrum

Overview

- Agile framework used by most Evisions teams
- Organizes work into fixed-length sprints
- Focuses on predictable delivery cycles

Sprint Structure

- **Sprint Planning** – team selects work from the backlog; goals established for the sprint

- **Daily Stand-Up** – team members discuss what they completed, what they will work on next, and any blockers
- **Sprint Review** – demonstrate completed work; gather stakeholder feedback
- **Retrospective** – review successes, identify problems, determine process improvements

Scrum vs Kanban

- **Scrum** – fixed sprint cycles, defined delivery targets, predictable planning structure
- **Kanban** – continuous workflow, work pulled as capacity becomes available, better suited for interrupt-driven teams
- Most Evisions teams use Scrum; MAPS team uses Kanban

Backlog Management

Backlog

- **Purpose** – central repository for bugs, features, enhancements, and internal ideas
- Prevents ideas from being lost
- Creates visible priorities
- Reduces decision-making based on urgency alone

Grooming

- **Purpose** – prepare backlog items for development
- **Activities** – clarify requirements, estimate effort, define scope, prioritize work
- **Outcome** – tickets move to the Input Queue when ready for development

Workflow Stages

- Input Queue $\rightarrow$ Development $\rightarrow$ QA $\rightarrow$ Done

Bottlenecks

Identifying Bottlenecks

- **Time-in-Column Metrics** – measure how long work remains in a workflow stage; highlight areas causing delays
- **Daily Stand-Ups** – surface blockers quickly; prevent issues from growing unnoticed

Common Sources

- **Dependencies** – foundational work must be completed before dependent features begin (eg authentication systems, data infrastructure)
- **Scope Creep** – additional requirements introduced after development begins; causes delays and uncertainty

Continuous Integration (CI)

- **Definition** – automated testing process that runs whenever code is merged
- Detects regressions immediately
- Provides consistent testing environments
- Eliminates "works on my machine" issues
- Supports higher software quality
- **Relationship to QA:**
  - QA defines acceptance criteria
  - CI validates automated tests
  - Human QA evaluates scenarios automation cannot fully test

Breaking Down Work

Product vs Engineering

- **Product** – defines what should be built and why it matters
- **Engineering** – defines how it will be built and technical implementation details

Task Decomposition

- A single feature may become multiple implementation tasks
- Example – one feature broken into: video list section, refresh logic, creator panel, sidebar integration, category navigation
- **Best practices:**
  - Define scope before development begins
  - Move out-of-scope requests back to the backlog
  - Provide detailed requirements to reduce rework

Dashboard Design

KPI Tiles (Scorecards)

- **Purpose** – provide high-level summary metrics
- Display a single key measurement
- Provide immediate context
- Typically placed at the top of dashboards
- Common measures: Count, Count Distinct
- Example – Student Count scorecard using Count Distinct on Student ID

Count Distinct

- **Purpose** – counts unique values only
- Prevents duplicate records from inflating totals
- Example – a student appearing multiple times should still count as one student in a headcount metric

Info Cards

- **Purpose** – display detailed information about a selected record
- Example – selecting a student may display Student ID, Major, State, Ethnicity, Gender, Age

Dashboard Hierarchy

- **KPI Tiles** – answer: How many? How much?
- **Charts, Tables, and Info Cards** – answer: Why? Who?

Key Terms

- **Backlog** – prioritized collection of bugs, features, enhancements, and ideas
- **Input Queue** – tickets prepared and ready for engineering work
- **Acceptance Criteria** – conditions that must be met for a ticket to be considered complete
- **Scope Creep** – unplanned expansion of work beyond original requirements
- **Continuous Integration (CI)** – automated testing process that validates code changes during development
- **Sprint** – fixed-length Scrum work cycle, typically two weeks
- **Retrospective** – meeting used to evaluate and improve team processes
- **Blocker** – issue preventing progress on a task or ticket
- **KPI Tile / Scorecard** – dashboard component displaying a single aggregated metric

- **Info Card** – dashboard component displaying detailed information for a selected record
- **Dimension** – qualitative grouping field (eg Major, Ethnicity, Age Group)
- **Measure** – quantitative value being aggregated (eg Count, Sum, Average)
- **Count Distinct** – aggregation method that counts unique values only, ensuring accurate headcounts and reporting

6/9 – Argos X Stand-Up, Frontend & Backend Development, Deployment Models, AI & Software Engineering

Argos X Stand-Up

- Observed a live daily Scrum stand-up with the Argos X engineering team
- Each team member gave a brief status update: what they completed, what they are working on, and any blockers
- Out-of-scope discussions were deferred to the "parking lot" section at the end to keep the meeting efficient
- Parking lot items included: a Data Agent structural refactor, a permissions demo, and a bug report around Invoke connections converting numerical values to Boolean in SQL QuickOpen
- Key takeaway: stand-ups are intentionally short and focused – rabbit holes kill the incentive to attend

Backend Development

Key Components

- **Server** – receives requests and executes business logic to return the expected result
- **Database** – stores application data including user accounts, configuration, content, and user-generated data
- **APIs** – allow software systems to communicate with each other

Core Concepts

- **Server-side scripting languages** – C++, JavaScript, TypeScript, Python, Java, PHP, Ruby; languages provide the foundation while frameworks (eg Next.js) provide tools and structure on top
- **CRUD operations** – the four basic database interactions: Create, Read, Update, Delete; used to store and manage user state (eg saving theme preferences, account info)
- **Authentication & authorization** – methods for verifying user identity and permissions; common approaches include JWT, OAuth, and session-based auth; ensures only authorized users can access the software
- **Security practices** – protecting against vulnerabilities like SQL injection and XSS (cross-site scripting); includes encrypting passwords and validating all inputs before they reach the database

Frontend Development

Key Components

- **HTML** – provides the structure of a web page; defines the hierarchy of elements and how they nest
- **CSS** – handles styling; controls colors, layout, and visual appearance
- **JavaScript** – handles dynamic and interactive content (eg animations, games, real-time updates); renders and responds to user actions

Core Concepts

- **Frameworks & libraries** – tools that prevent building from scratch; React is the current standard at Evisions and the most widely adopted framework today; Angular was dominant ˜10 years ago
- **Responsive design** – ensuring applications work correctly across various devices and screen sizes
- **Component-based architecture** – building UIs from reusable components so elements like buttons or links do not need to be rebuilt every time they are used
- **State management** – tracking and preserving what a user was doing or had configured (eg theme, last page visited) so it persists across sessions
- **Client-side routing** – ensuring that user actions on the frontend (like button clicks) are correctly routed to the appropriate backend endpoint

APIs – Frontend & Backend Intersection

- APIs are the communication layer between frontend and backend
- Frontend sends a request to the backend API (eg user clicks a button); backend processes it and returns data; frontend displays the result
- Authentication tokens are passed between frontend and backend to verify identity and maintain login state across pages
- Server-side rendering vs client-side rendering – determines whether the server or the browser assembles the final view; affects what different users see based on their permissions

Deployment Models

On-Premises (On-Prem)

- Software is installed and runs on computers physically located at the organization
- Organization is responsible for managing and maintaining its own hardware, servers, and networking
- Higher upfront cost (hardware purchase), but lower ongoing fees over time
- More direct control over customization and data; security depends entirely on the organization's own IT capabilities

Cloud

- Software is hosted on servers owned by a third-party cloud provider and accessed via the internet
- Provider manages storage, computing power, and networking
- Lower upfront cost; ongoing subscription or usage fees
- Modern cloud providers often have stronger security than an organization's internal IT team, since cybersecurity is their core business

Hybrid

- Combines on-premises infrastructure with cloud services
- Data and applications can be shared between local and cloud environments
- Requires managing both environments simultaneously
- Evisions' Argos runs on a hybrid model – institutions maintain on-prem installations of Maps/Argos, but users can access reporting via Argos X and Argos WebViewer in the cloud
- IntelleCheck and FormFusion are fully on-premises products

How Deployment Models Compare

- **Cost** – on-prem is higher upfront but potentially cheaper long-term; cloud has lower startup costs but ongoing fees
- **Control** – on-prem gives the organization full control; cloud shifts control to the provider
- **Scalability** – cloud is generally easier to scale; on-prem is limited by physical hardware capacity
- **Security** – on-prem security depends on internal IT; cloud providers often outperform internal teams due to specialization
- **Customization** – on-prem typically allows more customization; cloud customization depends on what the provider supports

AI & Software Engineering – AI Guardrails

Why Guardrails Are Necessary

- AI is powerful and useful but always makes mistakes – this problem will not fully go away
- AI always produces an answer even when it lacks the right context; it does not refuse or stay silent; hallucinations are the result
- Human review is essential for catching errors before they cause real-world consequences

Core Guardrails

- **Human in the loop** – all AI-generated content must be reviewed by a human before use; never trust output blindly or pass it along without checking
- **Visibility of AI content** – always know which content was generated by AI; use tools that visually flag AI output so you are never unaware of what came from a model vs a human
- **No autonomous action on high-risk operations** – AI should never independently execute actions with serious consequences (eg auto-replying to a professor's email, making financial trades); always generate and review, never auto-act
- **Autonomous action only when safe** – if allowing AI to act independently, the action must be reversible, involve a human approval step, or be extremely low-risk; acceptable failure = acceptable autonomy

How to Integrate AI Effectively in an Organization

- **Boost human creativity, do not replace it** – AI is not creative; it amplifies and supports human ideas rather than originating them
- **Increase human productivity** – use AI to work in parallel (eg spin up an agent during a meeting, review output after); amplifies individual output without replacing judgment
- **Replace tedious processes** – AI excels at repetitive, necessary-but-low-value tasks; frees up people for higher-order thinking
- **Improve communication and coordination** – AI can help keep stakeholders informed and up to date without requiring significant manual effort
- **Connected tissue, not a replacement** – the right mental model is AI as connective tissue between humans; it enables individuals to do more and connects teams better, rather than substituting for human roles

Positive Impacts of AI (Engineering Perspective)

- Eliminates most challenging or tedious code – describe the input, output, and behavior; AI implements it; developer reviews and adjusts
- Eliminates the blank slate problem – AI generates a starting proof of concept to react to and evolve, even if it is wrong initially

- Accelerates documentation – keeping docs in sync with code is dramatically easier
- Increases critical thinking time – because AI handles the doing, more mental energy goes toward reviewing, evaluating, and problem-solving
- Enables small teams to punch above their weight – a 10-person team coordinating with AI can operate with the output capacity of a much larger team
- Opens economically marginal verticals – products that previously could not justify a full team are now viable with AI-assisted development

The Future of AI in Software (Trevor's Take)

- Structured form inputs (dropdowns, text fields, date pickers) will become secondary; natural language chat and voice will become the primary way users interact with software
- Non-technical product stakeholders will increasingly be able to build and visualize their own solutions without going through engineering
- Engineering's role shifts from building features to ensuring AI-generated code is safe, correct, and production-ready
- Vision: customer feature requests go directly to an AI agent for implementation, bypassing the traditional months-long ticket-to-release cycle – with engineering reviewing output rather than writing it from scratch

Key Terms

- **Backend** – the server-side layer of an application; handles business logic, database interactions, and authentication; not visible to the user
- **Frontend** – the client-side layer of an application; everything the user sees and interacts with in a browser or app
- **API (Application Programming Interface)** – a defined interface that allows different software systems to communicate with each other
- **CRUD** – the four basic database operations: Create, Read, Update, Delete
- **JWT (JSON Web Token)** – a compact token used to securely transmit authentication information between frontend and backend
- **OAuth** – an open standard for token-based authentication and authorization; allows third-party login (eg "Sign in with Google")
- **SQL Injection** – a security vulnerability where malicious SQL code is inserted into an input field to manipulate a database; prevented through input validation
- **XSS (Cross-Site Scripting)** – a security vulnerability where malicious scripts are injected into web pages viewed by other users
- **React** – a JavaScript framework for building component-based user interfaces; currently the most widely adopted frontend framework; used by Evisions for Argos X
- **Responsive Design** – designing UIs to work correctly across different screen sizes and devices
- **State Management** – tracking and persisting the condition of a user's application session across interactions and page navigations
- **On-Premises (On-Prem)** – a deployment model where software runs on hardware owned and maintained by the organization itself
- **Cloud Deployment** – a deployment model where software is hosted and managed by a third-party provider and accessed via the internet
- **Hybrid Deployment** – a deployment model that combines on-premises infrastructure with cloud services; Evisions' Argos uses this model
- **Hallucination** – when an AI model generates a confident-sounding answer that is factually incorrect or not grounded in its actual context; results from the model always producing

output even when context is insufficient
- **Human in the Loop** – the principle that a human must review and approve AI-generated output before it is acted upon or published
- **Proof of Concept (POC)** – a quick, rough implementation used to test whether an idea works and to give stakeholders something tangible to react to; not production-ready
- **V0 (v0.app)** – a tool by Vercel that generates functional web applications from natural language prompts; used to rapidly prototype UIs without writing code
- **Vercel** – a cloud hosting company used by Evisions for its newer products; also provides an AI gateway for accessing inference models within applications

6/10 – Engineering Roundtable & QA + Exploratory Testing

Engineering Roundtable

- Observed Evisions' monthly all-engineering meeting; interns attended as guests
- Agenda covered org updates, AI/Copilot cost changes, and product demos from multiple teams
- Key takeaway: cross-functional visibility across siloed teams is rare – this meeting exists specifically to bridge that gap

Intro to Software Quality Assurance (SW QA)

What QA Actually Is

- QA is not just testing – it encompasses the processes and methods used to improve quality throughout the entire development cycle
- Testing alone is quality control (catching defects); QA is quality assurance (preventing them earlier)
- QA is involved from design through release, not just at the end
- Earlier defect discovery means cheaper fix; a bug found in production can take down a customer's system and costs significantly more to resolve
- QA also owns compliance: security certifications and accessibility requirements (eg screen reader support, high-contrast compatibility) are required by many clients
- Thankless by nature – users never credit QA when software works, but blame it first when something breaks

Common Testing Activities

- **Functional testing** – verifies that a specific feature meets the acceptance criteria defined in its ticket, plus implicit requirements like accessibility and keyboard navigation
- **Integration testing** – tests how multiple features interact with each other as a workflow or end-to-end scenario (eg log in → navigate to data block → use dashboard designer → save); ensures the system works as a whole, not just feature by feature
- **Platform compatibility testing** – validates software across all supported operating systems, browsers (Chrome, Firefox, Safari, etc), and device sizes (desktop, tablet, mobile); web apps use browser simulation tools for device testing; automated tests run across multiple screen sizes and browsers
- **Performance & scalability testing** – establishes response time baselines; validates that performance does not degrade as more users log in simultaneously
- **Security testing** – checks for known vulnerabilities that could expose or compromise data
- **Acceptance testing** – internal stakeholders (product, support, sales, marketing) and sometimes beta customers review the feature to confirm it matches what was intended; feedback

either feeds into the current release or becomes future requirements
- **Documentation review** – QA verifies that customer-facing instructions are accurate, typo-free, and clearly written with correct screenshots before release

Test Automation

- Modern agile development requires fast feature turnaround – manual testing alone cannot keep up
- Functional, integration, performance, and scalability tests are all heavily automated
- A large part of the QA role today is building and maintaining the test automation framework, not just running tests
- Automation handles standard regression testing so QA engineers can focus their manual time on exploratory and acceptance work
- **Continuous Integration (CI)** runs automated tests on every code merge to catch regressions immediately

Manual Testing

- **User Acceptance Testing (UAT)** is always manual – it simulates how a real user interacts with the software; automation cannot replicate genuine human use
- **Exploratory testing** is also manual – it investigates unexpected behavior that automation would not be looking for, because automation only checks what it is told to check
- **Ad-hoc testing** – informal, unstructured testing with no defined goal, no time-box, and no documentation; not recommended because when a bug is found, there is no log of what was done and results are nearly impossible to reproduce; used as a contrast to explain why exploratory testing is better

Exploratory Testing

What It Is

- A methodology where the tester defines a clear objective, then simultaneously learns the feature, designs tests, and executes them, all within a single focused session
- Not random testing – guided by a specific goal; every action is intentional
- Time-boxed, typically to one hour per session, to prevent rabbit holes and keep results sharp
- Shorter, focused sessions produce more accurate results than long open-ended ones
- Distinguishes itself from ad-hoc testing by requiring a documented execution log – what was touched, what was found, and in what order – so bugs can be reproduced and tracked

Charter

- A charter is the mission statement for an exploratory testing session – a brief, clear goal that keeps the tester focused and prevents aimless wandering
- **Standard format:** Explore [target feature] with [specific inputs/scenario] to discover [what you are looking for]
- **Example:** Explore the profile picture upload feature with images of various sizes and file types to discover any abnormal system behavior
- A single charter might test different file types (JPEG, PNG, TIFF), sizes (small, normal, over the limit), and behavior across all supported platforms (desktop, mobile, Chrome, Firefox, Safari)
- If something suspicious is noticed outside the charter's scope, note it and spin up a new charter; do not diverge mid-session
- When a session ends, new charters are generated from anything that seemed off; this is how coverage compounds over time

Best Practices

- Always adopt a user persona before starting – novice user, expert user, a specific role – and execute the session from that perspective
- Ask product what the feature is for and who the target user is before writing a charter; without that context, the test is guessing
- Stay within the charter's scope – if something out of scope looks broken, log it and move on; do not chase it
- Document the session: record screen and narrate actions; log all assumptions made (assumptions carry risk); produce an execution log for compliance purposes
- File all bugs found as Jira tickets – never just tell a developer verbally; verbal reports get lost and bugs leak to production

The Tourist Metaphor (James Whitaker)

- Pioneered by James Whitaker (former testing director at Microsoft and Google)
- Equates testing software to a tourist exploring a new city – both require a plan, both can wander without one
- Divides software into six "districts," each representing a type of area to explore:
  - **Business District** – the core money-making features of the product
  - **Historical District** – legacy code or areas known to be historically buggy
  - **Tourist District** – areas new users naturally gravitate toward first
  - **Entertainment District** – supporting features that are not flashy but are regularly used
  - **Hotel District** – background features that run while the software is not actively used (eg scheduled backups)
  - **Bad Part of Town** – areas where bad actors might attempt to exploit vulnerabilities
- Each district maps to a type of charter ("tour") a tester can run

Sample Tours

- **Guidebook Tour** – follow the user manual step by step as a new user would; simultaneously exercises features and validates that documentation is accurate
- **Money Tour** – model the test session against the sales or marketing demo; ensures the demo always works and does not embarrass the sales team
- **Landmark Tour** – exercise the major money-making features in a logical sequence
- **Intellectual Tour** – push the system with extreme inputs; example: a pie chart designed for 13 items gets fed 300 to see if it breaks; stress-tests boundaries and edge cases
- **Supermodel Tour** – purely visual; verify that the UI matches UX design specs across all supported platforms; checks color, font, layout, and appearance without focusing on functionality
- **Souvenir Tour** – visit every part of the software that generates an output (reports, exports, downloads) and collect them; verifies they all work correctly
- **Antisocial Tour** – act as a bad actor; enter illogical, malicious, or invalid inputs to see if the software can be broken or exploited
- **Couch Potato Tour** – use only default inputs and click through everything passively; tests that the out-of-the-box experience works without any customization
- Teams can also invent and name their own tours based on their product's specific needs

Key Terms

- **Software Quality Assurance (QA)** – the discipline of embedding quality throughout the entire development lifecycle, not just verifying it at the end
- **Functional Testing** – verifying that a specific feature works as defined by its acceptance criteria
- **Integration Testing** – testing how multiple features or systems work together as end-to-end workflows
- **Platform Compatibility Testing** – validating software behavior across different operating systems, browsers, and device sizes
- **Performance Testing** – measuring response times and validating that performance holds under increased user load
- **Security Testing** – identifying vulnerabilities in the software that could be exploited to expose or compromise data
- **User Acceptance Testing (UAT)** – manual testing performed by internal stakeholders or beta customers to confirm the feature delivers what was intended
- **Ad-hoc Testing** – unstructured, undocumented testing with no defined goal or time-box; produces unreproducible results; avoided in favor of exploratory testing
- **Exploratory Testing** – a structured manual testing methodology where the tester simultaneously learns, designs, and executes tests guided by a time-boxed charter
- **Charter** – a brief mission statement for an exploratory testing session; format: Explore [feature] with [inputs] to discover [outcome]
- **Test Log / Execution Log** – documentation of what was tested during a session, what inputs were used, and what was found; required for compliance and bug reproduction
- **Regression Testing** – re-running previously passing tests after new code is added to ensure nothing that worked before has broken
- **Smoke Tests** – a lightweight subset of automated tests run on every pull request to catch obvious breakage quickly without running the full test suite
- **PIDM** – a unique student identifier used in Banner and other Ellucian SIS platforms; the correct field to group on when counting students in a dataset
- **Acceptance Criteria** – the defined conditions a feature must meet to be considered complete and ready for release

6/11 – Test Automation Demos & AI in Testing

Maps Automation

- Maps = Multiple Application Platform Server; Windows service that underpins all Evisions products (Argos, FormFusion, IntelleCheck); described as "electricity" – changes to Maps cascade to every application
- Testing approach requires both UI and API automation; Windows-based apps introduce unique challenges since most modern automation tooling (Selenium, Playwright) targets web apps, not native executables

Maps UI Automation (TestComplete)

- TestComplete is a licensed (non-open-source) IDE and test library for Windows application automation
- Test execution flow: tear down existing Maps install → fresh setup → run tests → clear artifacts → uninstall

- Name mapping: maps Win32 UI objects to variables usable in test scripts
- Event controller handles runtime errors (stop on error, ignore and continue, etc)
- Scripts split into local (feature-specific tests) and shared (reusable functions across products)
- UI automation is slower and more fragile than API testing; rendering/color/object visibility issues can only be caught at the UI layer, that is the tradeoff that justifies maintaining both

Maps API Automation

- Uses Vitest; fully headless, parallel, open-source stack
- Worker count scales with database type: Postgres supports many concurrent transactions; SQLite is write-locked (one transaction at a time), so parallel workers are capped accordingly
- Tests are parallel vs serial depending on whether they modify server state; anything that takes Maps offline runs serially
- Teardown, Slack notification, and build artifact posting are all baked into the Jenkins pipeline

Jenkins & CI/CD

- Jenkins pipeline (Jenkinsfile) handles: fetch build artifacts → install EXEs → run tests → post results
- Ephemeral EC2 instances spun up from an AMI on each run; torn down after
- Maps automation is a downstream project of Maps/Argos builds; new build auto-triggers the automation suite
- Two CI/CD systems in use at Evisions: Jenkins (Maps suite) and GitHub Actions (Argos X / Fleet Manager)

AI in Maps Automation

- AI works well for API automation (open-source stack, familiar tooling)
- AI struggles with TestComplete: proprietary, non-open-source, legacy JS engine that does not support modern async patterns (await/promises)
- **Best practices shared:**
  - Create agents.md files in repos – short, precise instruction sets that tell AI agents what to do, what files to touch, and what standards to follow
  - Use the right model for the right task – do not use Opus for simple questions; use heavy models for planning, lighter models for execution
  - Plan mode + execute mode: use a capable model to generate a multi-step plan with small per-step context windows, then point a lighter model at execution
  - Context is critical – too little and AI has no idea what it is doing; too much and it loses coherence

Fleet Manager / Argos X Automation (Playwright + GitHub Actions)

- New product; QA started within the last few months; chose Playwright because Argos X already uses it successfully, shared tech stack means engineers can move between teams more easily
- Folder structure: qa/ → src/ (controls, pages, constants, helpers) and test/ (smoke, regression, accessibility, with desktop/mobile/shared subfolders)
- Locator factory files: every UI object gets a data-testId attribute; locator files map those IDs to variables used across tests, single source of truth for object mappings
- Page object files: wrap locator references in named, readable functions (eg searchForInstanceByName); keeps test scripts clean and readable
- Smoke tests: run on every Vercel deployment; ˜2–3 minutes; auto-posts pass/fail to the PR;

blocks promotion to production on failure
- Regression/accessibility/security: run on a schedule (daily) and can be triggered manually via GitHub Actions with granular browser/viewport/feature selection

AI in Argos X / Fleet Manager QA

- Significant AI usage across four workflows:
  - **Test plan generator** – AI scans the PR, produces a checklist of manual test cases including environment URL, preconditions, and step-by-step test cases
  - **Test executor** – AI agent logs into Argos X in a real browser and executes the test plan, posts results back to the PR
  - **Accessibility analysis** – scans code pre-QA, flags WCAG violations before they reach the team
  - **Test coverage analysis** – scans code, flags missing unit/component tests, calls out breaking changes that will fail existing automation
- Practical impact: by the time a ticket reaches QA manually, it is already been through multiple agentic layers; QA can focus on edge cases rather than basic coverage
- Prompt quality matters – vague prompts produce vague output; context has to be well-defined
- AI generates structure and logic confidently but still requires human review and validation

Localization & Accessibility (Sundar)

- Localization = adapting a product for different regions: language, date formats (MDY vs DMY), currency, phone formats, calendar start day, right-to-left rendering (Arabic, Hebrew)
- At Evisions, translations are handled automatically via AI through a service called Locize – English strings in code get pushed to Locize, translated, and returned to the app; theoretically supports any language
- QA does not need to speak the language; they are checking that English strings do not appear when a non-English locale is selected, and that UI renders correctly (text not overflowing, layout not breaking)
- Accessibility = ensuring software is usable without a mouse and by visually/hearing-impaired users; compliance standard is WCAG (Perceivable, Operable, Understandable, Robust)
- Keyboard navigation testing: tab, shift-tab, arrow keys must reach every interactive element in logical order
- Tools: AXE DevTools, WAVE, Lighthouse (browser), NVDA and JAWS (screen readers, Windows), VoiceOver (Mac)
- Axe/PAX scans run as part of the automated accessibility suite; violations get ticketed in Jira for developers to fix

Key Cross-Cutting Themes

- UI automation catches what API testing misses (rendering, visual defects, fragile UI states); both layers are necessary
- The open-source/proprietary split directly determines how useful AI is as a coding assistant
- Agent instruction files (agents.md) are a lightweight but high-leverage way to make AI effective in any codebase
- The QA role today is as much about building and maintaining automation infrastructure as it is about running tests

6/12 – Business, Communications & Philanthropy

Business

Company Structure

- $35M company size; typical C-suite is 4–5 leaders
- Standard roles: CEO/President, CFO (common once company hits ˜$25M), Chief Revenue Officer (sales/retention), Chief Tech/Product (split at Evisions: Jim Farrell = product, Jeff Beaman = engineering)
- Deliberate "checks and balances" – separating roles (eg sales vs marketing) creates healthy tension
- Three leadership layers:
  - **Operational managers** – day-to-day, direct reports, scheduling/coverage (eg Kirsten in support)
  - **VPs/Directors** – tactical, bridge between C-suite and managers
  - **C-suite** – strategy, big picture
- Culture: anyone can ask questions/contribute outside their area

Decision Making

- Constant "natural debate" among leadership – disagreement is healthy, leads to best path forward
- Balance is the recurring theme – almost every decision is a tradeoff (cost vs employee happiness, settle vs litigate, etc)

CFO Day-to-Day

- Oversees financial infrastructure: general ledger, billing, revenue recognition, expense systems
- Reviews customer contracts/changes daily
- Meetings ˜12pm–4pm: revenue sync (sales + marketing + finance), partnership briefings, 1:1 check-ins
- **Listening** is core to the role – picks up on risk signals (product obsolescence, team burnout, customer over-reliance on support) before they become bigger problems
- When something urgent comes up, internal meetings get sacrificed first (delegate or get Slack updates instead)

Contracts & Legal

- HR policies are legal documents – poorly worded policies create lawsuit exposure
- Settle vs litigate = cost-benefit balance (eg $100K to litigate vs $10K to settle, but settling can invite more claims)
- Contract terms protect predictable revenue: no "terminate for convenience," fixed payment terms (vs customers wanting 90-day terms)
- Choice of law clauses are negotiable depending on risk level

Tackling Customer Issues

- Understand the facts first before responding (was it our fault)
- Example: customer hit by typhoon, could not pay – company chose to extend service for free but built in a check-in date to potentially shorten the grace period (balance between helping customer and minimizing cash exposure)

Financial Levers/Metrics

- Leading indicators: sales pipeline (reviewed weekly)

- Lagging indicators: headcount, revenue, churn
- Vendor contract review – at least twice a year, check if services still add value (cost creep is common)
- 3–5 year forecasting with constant scenario analysis ("what if" planning)

HR, Benefits & Compensation

- People = largest cost line item
- Total compensation = pay + benefits (health insurance ˜$1.2M/year, company covers ˜80%)
- Payroll taxes = 8% of pay
- 401k compliance = regulated by Department of Labor/ERISA; strict deadlines for remitting employee contributions

Compensation Strategy Example – Medical Renewal

- Annual ˜10–12% premium increases from carriers
- Negotiate down, then decide: absorb the cost, pass it to employees, or switch carriers/plans
- Requires careful change communication so employees understand the "why"

Risk Management & Insurance

- 6 types of insurance carried (eg general liability, D&O)
- Strict reporting windows (eg 24 hours to notify carrier of a claim/demand letter) or risk losing coverage
- Process: notify carrier → investigate → respond to counterparty → decide settle vs litigate

Operations

- Manages workflows/processes behind the business: contract operations, billing automation, software purchasing/implementation, vendor negotiation, security/deployment

M&A

- Reasons to pursue: acquire talent/teams, enter new markets, complementary tech, leverage existing relationships, defensive (remove competitor), investor exit timing
- Stages: identify target → confidentiality agreement → financial review/valuation → due diligence (legal, tax, technical) → negotiate purchase agreement (300–400 pages) → sign → integrate
- Tied to **Total Addressable Market (TAM)** – assessing how much market is left to capture

Communications

Business Stack (CEO/Executive Networking)

- Designed around appealing to ego – 7 visual anchors:
  - Welcome mat ("HOW") → How did you get started in business
  - Headquarters → Why did you start the business here
  - Floor covered in change → How has the business changed since you got into it, what is coming
  - Room with teacher/blackboard (lessons) → What is the biggest lesson you have learned
  - Room with Olympic podium (successes) → What are you most proud of, greatest achievement
  - Room with weightlifters (challenges) → What is your biggest challenge right now
  - Room with soccer goal (goals) → What are you focused on/working toward

Mirroring & Restating

- **Mirroring** – repeat 1–2 of the other person's words back (especially emotional ones); they will naturally fill the silence and elaborate
- **Restating** – summarize back what they said ("sounds like you...") to prove you are listening; builds trust, encourages them to share more
- Use sparingly – overuse becomes annoying

Support, Do Not Shift

- **Shift** = redirecting conversation back to yourself (happens ˜73% of the time naturally)
- **Support** = staying with what the other person said, digging for more
- Even if you disagree/have no shared interest, support keeps rapport ("oh good for you, what do you like about that")
- People trust those who take interest in them – supporting builds likability regardless of personal opinion

Philanthropy

Inner Workings / Definitions

- Time, treasure, talent – the three forms of giving
- **Bequest** – leaving money/assets to charity in a will

History

- Traces back to Plato (347 BC, funded an Academy via will)
- Harvard founded 1638 via John Harvard's bequest

Why Philanthropy Matters

- Strengthens communities, addresses inequality, improves education/health, responds to crises
- Studies link philanthropy to lower depression, higher self-esteem, lower blood pressure, longer life
- 88% of consumers prefer purpose-driven companies; 60%+ of Gen Z/Millennials want to work for socially responsible companies
- 2024 US giving: $592B total – 67% individuals, 19% foundations, 8% bequests, 7% corporations (corporate giving noted as lowest/most disappointing share)
- Giving destinations (by volume): religion, human services, education, public society benefit, health, foundations, international affairs, arts/culture, environment/animals, individuals (scholarships)

Nonprofits

- Definition: mission-driven org addressing social cause/community need; invests in "social profit" vs profit
- Four types: civic/political engagement, service delivery, expression of values/faith, social entrepreneurship
- Run like a business: board of directors, staff, salaries, financial sustainability, "businesses with a mission"
- Tax-exempt; donations are tax-deductible for donors
- Revenue sources: events/galas, grants, individual donors, membership fees
- Budget breakdown: majority → programming, some → fundraising, ˜10% → admin/overhead
- Strong nonprofit traits: clear mission, strong leadership, fundraising stability, community trust, measurable impact, good storytelling
- Fundraising = sales; must "sell" the mission to donors

- Notable philanthropist examples: MacKenzie Scott (large unrestricted grants), Riot Games (community-voted grants), Mr Beast (creative/viral giving), Vanessa Bryant (community investment)
- **Assignment**: "Philanthropist" project – research a philanthropist's giving strategy, financial contributions, and impact; due in two weeks, presented at capstone
- **Nonprofit Idol**: separate assignment – research a nonprofit (budget, EIN, mission, programs), propose how Evisions could support them; group votes on one to present at capstone with a possible surprise for the winning org